Greeks Engine: Preparation Guide for the Project Review

The project explained, the figures that go into the paper, and the questions to expect

Yash Agrawal · BITS Pilani

27 October 2026

Contents

1 The project in two minutes **1**

2 The mathematics you must be able to defend **2**
2.1 Heston model . . . 2
2.2 The COS method (Fang & Oosterlee, 2008) . . . 2
2.3 The Greeks we compute . . . 2
2.4 Adjoint (reverse-mode) differentiation . . . 2
2.5 Fixed point and the first-order error bound . . . 3

3 Architecture and how it works **3**
3.1 The generator (`hardware/gen/`) . . . 3
3.2 Inside the engine . . . 4
3.3 Three configurations (never mix their numbers) . . . 4
3.4 Board systems . . . 4

4 The figures in the paper and how to explain them **4**

5 Results with the honest numbers **8**

6 Limitations and status (say these before he finds them) **8**

7 Repository map **9**

8 Questions the supervisor may ask, with answers **9**
8.1 Novelty and positioning . . . 9
8.2 Heston and COS . . . 9
8.3 Adjoint differentiation . . . 10
8.4 Fixed point and the bound . . . 10
8.5 Hardware . . . 11
8.6 Results and fairness . . . 11
8.7 Practicalities and status . . . 12

9 Last-minute advice **12**

1 The project in two minutes

What it is. An FPGA engine that computes the price of a European option under the **Heston stochastic-volatility model** together with **all nine sensitivities (Greeks)** in one pass, using **adjoint (reverse-mode) differentiation** built directly into a fixed-point hardware datapath.

Why it matters. Greeks drive hedging and risk. The industry default on hardware is *bump-and-reprice*: change one input, price again. Nine Greeks by central differences cost 19 pricings, and their accuracy depends on an arbitrary bump size. In software, adjoint differentiation is the standard alternative, but it records a "tape" and costs about 5–8 pricings in practice. **No published FPGA work computes Greeks by an adjoint.**

The key idea. The COS pricing method performs exactly the same operations for every input. So the reverse sweep can be written once, scheduled at design time into the same datapath as the forward pass, with no tape memory and no replay logic, and its multiplications fill multiplier slots the forward pass leaves idle.

Headline results.

- Price + 9 Greeks for **1.04**× the cost of the price alone (4,733 vs. 4,568 cycles); bump-and-reprice needs 18.4× more cycles on the same hardware.
- A **first-order rounding-error bound** for every Greek, never exceeded: worst 0.378 of the bound over 10,500 inputs, 0.356 on a held-out 10,500, ≤ 0.255 on silicon.
- **Bit-exact on two FPGA families**: ZedBoard (Zynq-7020) 450/450; ZCU102 (Zynq UltraScale+) 1,010/1,010, including 50 real NIFTY market calibrations.
- **One generator** produces everything from one description: bit-accurate model, error bound, scheduled Verilog, bus wrappers and host C code.

2 The mathematics you must be able to defend

2.1 Heston model

Under the risk-neutral measure: $\mathrm{d}S = rS\,\mathrm{d}t + \sqrt{v}\,S\,\mathrm{d}W_1$, $\mathrm{d}v = \kappa(\theta - v)\,\mathrm{d}t + \xi\sqrt{v}\,\mathrm{d}W_2$, $\mathrm{d}W_1\mathrm{d}W_2 = \rho\,\mathrm{d}t$.

Nine inputs: S_0 spot, K strike, T maturity, r rate, v_0 initial *variance*, κ mean-reversion speed, θ long-run variance, ξ vol-of-vol, ρ correlation. No closed-form price, but the **characteristic function** of $\log S_T$ is known. We use the numerically stable form (Albrecher et al., "the little Heston trap"), which avoids the branch-cut discontinuity of the complex logarithm.

2.2 The COS method (Fang & Oosterlee, 2008)

Truncate the log-price density to $[a, b]$, expand it in a cosine series whose coefficients come straight from the characteristic function:

$$V \approx Ke^{-rT} \sum_{k=0}^{N-1}{}' \mathrm{Re}\Big[\varphi(u_k)\, e^{iu_k(x-a)}\Big]\, U_k, \qquad u_k = \frac{k\pi}{b-a}.$$

Our choices: $N = 128$ terms; range $c_1 \pm 10\sqrt{c_2}$ from the cumulants ("±10 standard deviations"); the range is **held fixed** when differentiating, so the Greeks are exact derivatives of the COS approximation at a fixed range (the reference uses the same convention). We **price puts and get calls by put–call parity** $C = P + S_0 - Ke^{-rT}$: call payoff coefficients grow like e^b and cancel badly in fixed point; parity made the worst call price error 75× smaller. **The 128 terms are independent of each other**: the hardware pipelines them.

2.3 The Greeks we compute

Delta $\partial V/\partial S_0$; strike sensitivity $\partial V/\partial K$; theta $\partial V/\partial T$; rho $\partial V/\partial r$; vega $\partial V/\partial v_0$ (with respect to *variance*; $\partial V/\partial \sigma_0 = 2\sigma_0\,\partial V/\partial v_0$); and the model-parameter sensitivities $\partial V/\partial\kappa$, $\partial V/\partial\theta$, $\partial V/\partial\xi$, $\partial V/\partial\rho$, used in calibration and model risk.

2.4 Adjoint (reverse-mode) differentiation

Forward (tangent) mode pushes one input direction through the computation: one sweep per input, nine here. Reverse (adjoint) mode pushes the sensitivity of *one output* backwards and yields all input

derivatives at once; the "cheap gradient principle" says this costs a small constant multiple of the function (theory: $\leq$ 4–5×).

In software the forward values needed by the backward sweep are stored on a **tape**, which is memory-bound: 5.7–7.6 pricings on our CPU (CoDiPack 7.6). In our engine there is **no tape**: control flow never depends on the input, so every forward value's lifetime is known at design time, and the scheduler keeps it in a register exactly as long as the reverse sweep needs it (447 storage registers per term). It is *the tape compiled into the schedule.*

State this yourself: the reverse sweep is **hand-derived** as operations in the generator's graph (`heston.py`) and verified numerically. The generator does not differentiate automatically yet. The claim is "adjoint differentiation", not "*automatic* adjoint differentiation"; "aad" in module names is historical.

2.5 Fixed point and the first-order error bound

Every value is an integer $\times 2^{-FL}$: the ZedBoard design uses 56-bit words with 28 fraction bits (Q28), the 64-bit design 64-bit words with 32 (Q32). Round-to-nearest after every operation, so each rounding contributes at most $\frac{1}{2}$ ulp $= 2^{-(FL+1)}$.

The bound (Linnainmaa 1976, first order):

$$|\text{error in output } y| \leq \sum_{\text{rounding nodes } i} \left|\frac{\partial y}{\partial w_i}\right| \cdot 2^{-(FL+1)}.$$

The sensitivities $\partial y/\partial w_i$ come from a reverse sweep over the graph. Products of two rounding errors (second order, $\approx 2^{-28}$ smaller) are dropped. It covers **rounding only**, against the same algorithm computed exactly; the COS method's own truncation error is a separate term. The bound is computed **in software** by the model (0.1 s per parameter set); the hardware does not output it. Two checks justify dropping second-order terms: a 50-digit evaluation (hardware error ≤ 0.24 of the bound) and the 44-bit design (second-order terms 4,096× larger), where the bound still held.

3 Architecture and how it works

3.1 The generator (`hardware/gen/`)

Module	Job
`heston.py`	Builds the operation graph: setup (once), term k (×128) including the reverse sweep, finish (once). Bit-accurate emulator.
`ir.py`	Graph representation; fixed-point and double evaluation; the error bound.
`prims.py`	Newton–Raphson reciprocal and sqrt (table seeds), Tang's exponential, log (table + series), CORDIC rotation/vectoring. No dividers.
`ranges.py`	Measures the data-dependent shift ranges over the input domain (with margin); shifters built only that wide.
`sched.py`	Modulo scheduler: shares 8 multipliers and 5 CORDIC units across overlapping terms.
`rtlgen.py`	Emits the Verilog.
`wrappers.py`	AXI4-Lite / AXI-Stream register-file wrappers.
`host.py`, `cgen.py`	Host setup/finish steps in Python and as bit-exact generated C (ARM).
`tangent.py`	The three forward-mode baselines (dense, sparse, factored).
`check_accuracy.py`, `board_vectors.py`, `chain.py`	Accuracy checks, board test vectors, strike-chain study.

Because the RTL and the model come from the same graph, the testbench checks **every register at the cycle it becomes valid** (1,213–1,962 checks per design), so the hardware inherits every property shown for the model.

3.2 Inside the engine

- Per evaluation: **setup** once (characteristic-function constants, truncation range), **128 terms** (forward pass, then its reverse sweep), **finish** once (discounting, final chain rule, put–call parity).
- Per term: forward pass (payoff coefficient, characteristic function) → reverse sweep back to the inputs (reusing the forward reciprocals) → power-of-two normalisation (keeps tiny adjoints of high-frequency terms at full precision) → nine accumulators.
- Each term costs exactly **250 multiplications, 3 CORDIC rotations, 2 CORDIC vectorings** (the price alone: 126 multiplications).
- Resources: 8 shared multipliers, 3 rotation and 2 vectoring units. Initiation interval: $\lceil 250/8 \rceil = 32$, $3 \cdot 32/3 = 32$, $2 \cdot 32/2 = 32$: all three balance at **II = 32**. One term lasts 506 cycles, so about 16 terms are in flight. Loop: $127 \times 32 + 506 + 2 =$ **4,572 cycles**; fully on chip (`z7`): **4,733**.
- **Why the Greeks are nearly free:** the CORDIC units set the pace, so in every 32-cycle window the forward pass leaves multiplier slots idle; the reverse sweep's multiplies fill them.
- A 56×56-bit multiplier is built from 9 DSP48 blocks (25×18 each): 8 multipliers = 72 DSPs.
- Shifters cut to measured ranges saved 35% of logic; an input that would exceed a range raises `range_err` instead of returning a wrong value; no valid input raises it after the refit at the domain edges.

3.3 Three configurations (never mix their numbers)

Name	What	Cycles	Where it ran
`z7`	56-bit, everything on chip	4,733	simulated (too large with setup/finish for the Zynq-7020)
`z7h`	56-bit, setup and finish on the host, loop on the FPGA	4,572 (65.3 μs at 70 MHz)	**ZedBoard silicon**
`zu`	64-bit, 32 multipliers, everything on chip	1,593 (15.9 μs at 100 MHz)	**ZCU102 silicon**

3.4 Board systems

1. **ZedBoard JTAG system**: Vivado drives the engine over the JTAG cable (JTAG-to-AXI master → AXI4-Lite), no processor software: **450/450 bit-exact**.
2. **ZedBoard processor system**: the ARM Cortex-A9 runs setup and finish as generated C and times each stage. Built; board run pending.
3. **ZCU102 JTAG system**: the 64-bit engine: **1,010/1,010 bit-exact**.
4. **ZCU102 processor system**: the Cortex-A53 times the bus transfers and reads the board's INA226 power monitors. Built (timing met, +1.772 ns). The lab board's DDR does not calibrate with Vivado's preset, so the program now runs from on-chip memory and the setup skips the DDR steps; **this run is still pending**.

4 The figures in the paper and how to explain them

The paper (`docs/paper/greeks_engine_ieee.pdf`) has nine figures, in IEEE style. Below, in order of importance, is each figure with how to read it and what to say.

Fig. 1 – Design flow (diagram)

One operation graph (setup / term / finish) branches into the bit-accurate model, the double-precision model, the error bound and the modulo scheduler → Verilog; the RTL is checked register by register, implemented in Vivado, run on two boards, and the board outputs are compared bit for bit with the model.

Say: Because everything comes from one graph, the hardware is correct by construction relative to the model, and the bound applies to the hardware.

Fig. 2 – One COS term and the modulo schedule (diagram)

Top: forward pass → reverse sweep → normalisation → 9 accumulators, one datapath; forward values held in 447 registers. Bottom: bars 506 cycles long starting every 32 cycles; about 16 overlap.

Say: There is no tape memory: the adjoint is part of the datapath. II = 32 is set equally by the multipliers and the CORDIC units, and that balance is why the Greeks cost 4%.

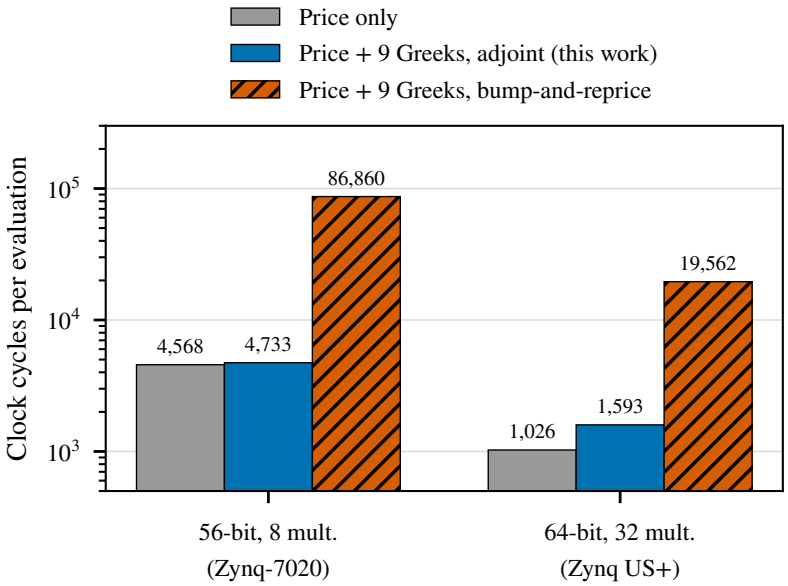


Fig. 3 – Cycles: price only, adjoint, bump-and-reprice on the same hardware.

How to read it: log scale. On the 56-bit design the adjoint bar (4,733) is almost level with the price bar (4,568), while bump-and-reprice is 86,860. On the 64-bit design the adjoint costs 1.55× the price because there the 32 multipliers, not the CORDIC units, set the pace.

Say: All nine Greeks cost 3.6% more than the price: 165 cycles. Bump-and-reprice needs 18.4× more cycles on identical hardware, and the generated design is 92× faster than our hand-written first generation.

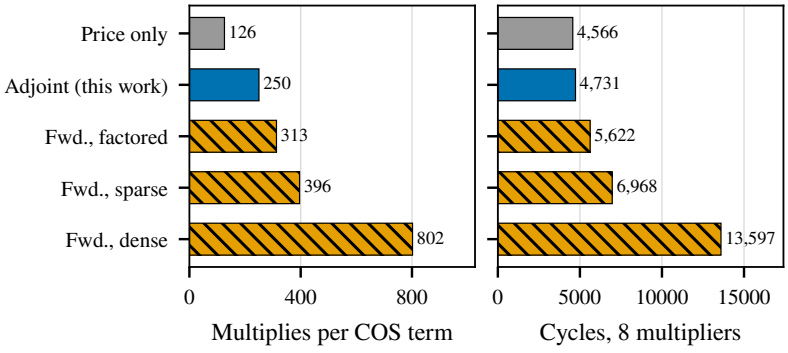


Fig. 4 – Adjoint against three forward-mode datapaths built by the same generator.

How to read it: left, multiplies per term (126 price, **250 adjoint**, 313 factored, 396 sparse, 802 dense); right, cycles with 8 multipliers (4,731 vs. 5,622, 6,968, 13,597). This is the answer to "why adjoint and not forward mode?".

Say: For the same Greeks the adjoint needs the fewest multiplies. To be honest: with 16 multipliers all but the dense one reach ≈4,570 cycles, so the 1.04× is not unique; what the adjoint saves is **hardware**: 8 multipliers (72 DSPs) vs. 10 (90) for the factored forward method and 26 (234, more than the chip has) for the dense one.

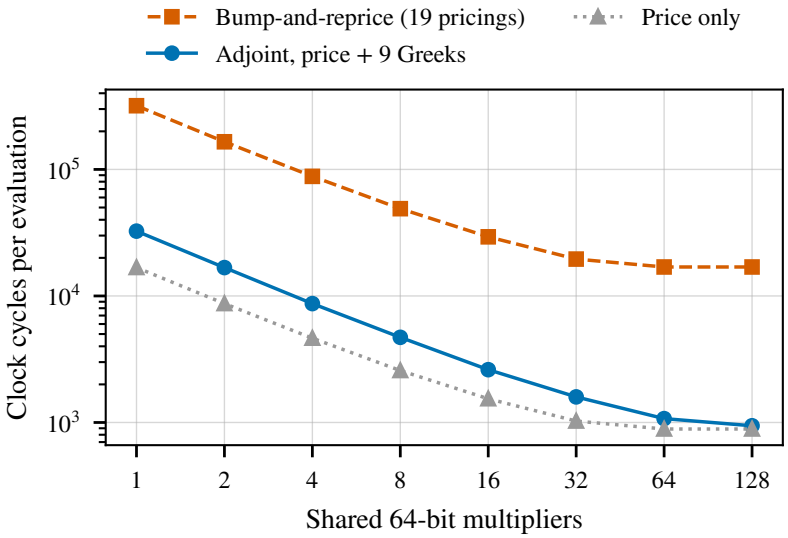


Fig. 5 – Cycles against the number of shared 64-bit multipliers.

How to read it: log–log; cycles halve as multipliers double until the dependency chain of a term dominates past 64. The adjoint stays 1.1–1.9× the price and 9.8–18× below bump-and-reprice throughout.

Say: One generator, one parameter: a small engine for a cheap FPGA or a fast one for a big FPGA, and the figure shows where extra hardware stops paying.

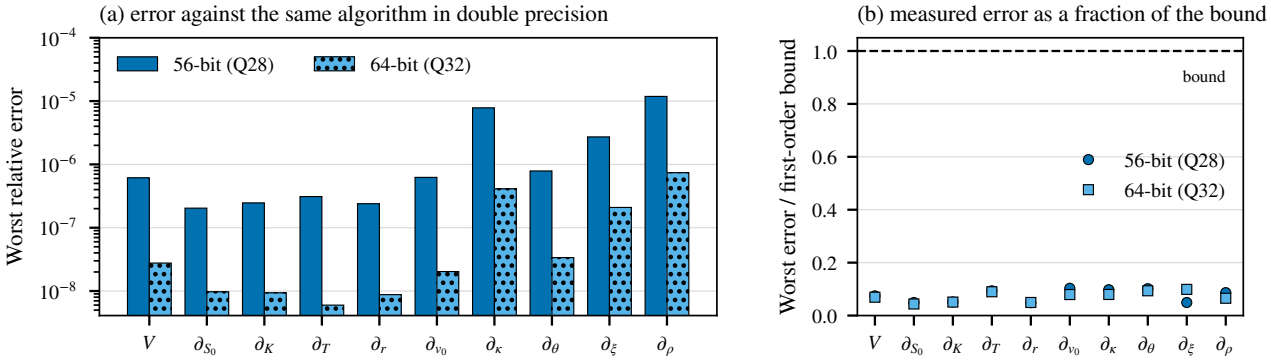


Fig. 6 – Accuracy over the 21-case grid: (a) worst relative error, (b) error / bound.

How to read it: (a) 56-bit errors 10^{-7} to 1.2×10^{-5}, 64-bit 10^{-8} to 7×10^{-7}; ∂_κ and ∂_ρ are the weakest. (b) every point is below 0.11, far under the dashed line at 1 (the bound).

Say: Every output stays below its bound; worst 0.103 on the grid. The bound is conservative, as a worst-case bound must be, but within a factor of ten.

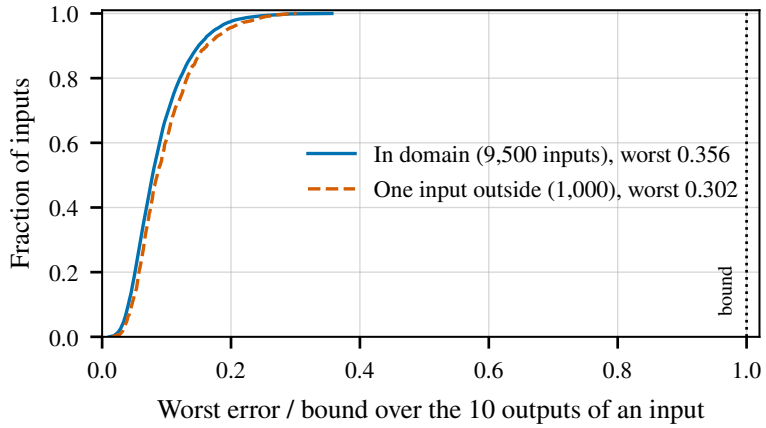


Fig. 7 – Held-out 10,500-input sweep: distribution of the worst error/bound per input.

How to read it: a cumulative distribution; the whole curve sits left of 0.36, nowhere near 1. Out-of-domain inputs that were not flagged also stay inside the bound.

Say: The bound was tested on inputs the shift ranges were never fitted to; no in-domain input raised `range_err` and none exceeded the bound.

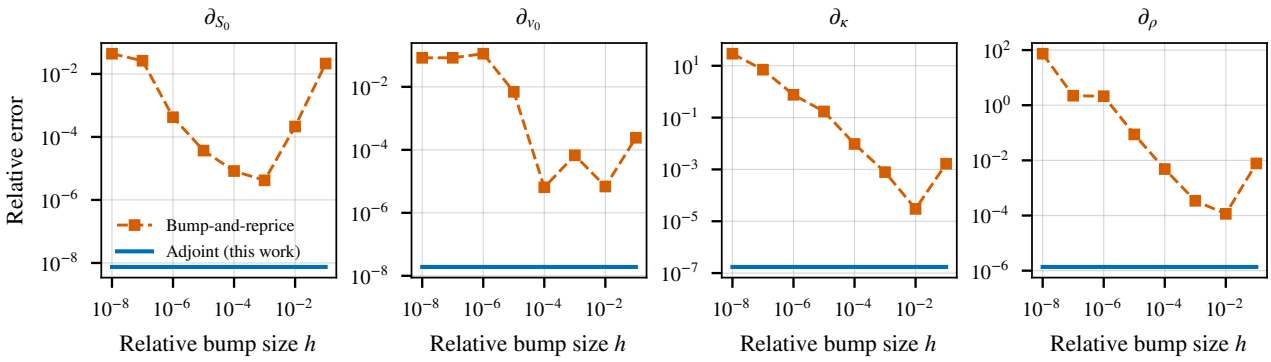


Fig. 8 – Bump-and-reprice error against bump size, and the adjoint (flat line).

How to read it: U-shaped curves: on the right the truncation error of central differences ($\propto h^2$); on the left rounding divided by h ($\propto \epsilon/h$), much worse in fixed point. The best h differs per Greek; the adjoint line is below every curve.

Say: The adjoint is 32–560× more accurate (median 145×) even when bump-and-reprice gets the best bump size chosen afterwards, and it has nothing to tune.

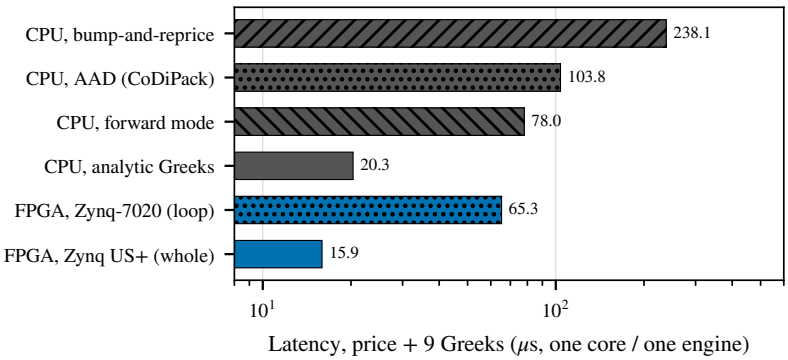


Fig. 9 – Latency of one price + 9 Greeks: one CPU core vs. one FPGA engine.

How to read it: log scale. The ZCU102 (15.9 μs) beats every CPU method, narrowly beating

analytic Greeks (20.3 μs); the ZedBoard (65.3 μs) beats the AD tools and bumping but not analytic Greeks.

Say: We are 4.9–15× faster than forward AD, taped AAD and bumping, 1.3× faster than hand-derived Greeks on the larger FPGA, and slower than them on the small one. Throughput is the CPU's: four cores give 2.5× one engine.

Tables in the paper

I: positioning against related work. II: the three configurations. III: implementation (LUT, FF, DSP, BRAM, clock, slack). IV: **assumptions with rationale** (A1–A14). V: adjoint vs. forward mode. VI: significant figures. VII: silicon validation (1,469/1,469 bit-exact). VIII: latency, throughput and energy vs. CPU (HWiNFO slots marked in red).

Not paper evidence

Vivado screenshots and the hedging figures (12 NIFTY trades) are in the report only. Hedging is an illustration: *"it shows the Greeks in use, not evidence for the hardware; hedge decisions do not depend on how the Greeks are computed."* The old fig1–fig5 in `validation/figures` are from the first-generation analysis and superseded.

5 Results with the honest numbers

Greeks cost	**1.04 pricings** (56-bit); 1.55 on the multiplier-bound 64-bit design
vs. bump-and-reprice	18.4× fewer cycles; 32–560× more accurate
Error bound	never exceeded over 10,500 + 10,500 held-out inputs (worst 0.378 / 0.356); 0.199 ZedBoard, 0.255 ZCU102
Accuracy (56-bit, 99th pct.)	price, delta, strike, theta, rho, vega $\geq$ 5.3 significant figures; ∂_κ 3.5, ∂_ξ 3.0
Silicon	ZedBoard 450/450 (+9/9 vector); ZCU102 1,010/1,010; worst ZCU102 price error 7.3×10^{-7}
Area	Zynq-7020: 72.8% LUT, 72/220 DSP; ZCU102: 77,515 LUT (28%), 510 DSP (20%), +1.601 ns at 100 MHz
Latency vs. CPU (i5-1155G7, idle)	analytic 20.3 μs; ZedBoard 65.3 (CPU 3.2× faster); ZCU102 15.9 (1.3× faster); 4.9–15× faster than forward (78.0), CoDiPack (103.8), bump (238.1)
Throughput	4 CPU cores 158k/s vs. one 64-bit engine 62.8k/s: **CPU wins**
Energy	44 μJ (ZedBoard), 40 μJ (64-bit), **simulated** activity; CPU pending HWiNFO (rated-power estimate 76–177 μJ, so 2–4×)
COS method error	separate from rounding; worst vega 7.8×10^{-3} ($\xi = 1$, $T = 0.34$); 256 terms fix it

Framing: the contribution is the **cost and certified accuracy of Greeks in hardware** and bit-exact silicon, not raw speed against hand-written CPU code.

6 Limitations and status (say these before he finds them)

1. The adjoint is **hand-derived**, not generated automatically.
2. The bound is **first order** and covers **rounding only**; COS truncation error is separate and unbounded.
3. **Energy**: FPGA simulated (SAIF), CPU pending measurement (HWiNFO).
4. The latency lead is **narrow** (1.3×) and only on the larger FPGA; the CPU wins throughput.
5. Bus transfers not yet timed (ZCU102 processor-system run pending).
6. Bounded input domain; S_0 fixed at 100 (prices scale with (S_0, K) together).
7. $\partial V/\partial\kappa$ and $\partial V/\partial\xi$: about 3 significant figures at the 99th percentile.
8. Zynq-7020 timing does not close at 100 MHz (Fmax ≈79; board at 70).
9. Hedging is an illustration only.

7 Repository map

- `README.md`, `dashboard.html` – overview and results dashboard.
- `docs/paper/` – the IEEE paper and its figure script; `docs/report/latex/` – the full report; `docs/architecture.md` – engineering notes with every number; `docs/novelty_assessment.md` – literature check and claim wording; `docs/greeks_engine_dossier.html`.
- `hardware/gen/` – the generator; `hardware/verilog/gen/` – generated RTL; `hardware/sim/` – primitive unit tests.
- `hardware/vivado/{zedboard, zcu104, zcu102, power}` – build and board scripts, SAIF power flow; `hardware/sw/{zedboard, zcu102}` – ARM programs.
- `validation/` – `reference/` (double-precision reference), `cpu_baseline/` (C++ and CoDiPack), sweep scripts, `results/` (every number and the board logs), `figures/`, `hedging/`.
- `verify_all.sh` – **26 automated checks**, all passing. "How do you know it is right?" – run it.

8 Questions the supervisor may ask, with answers

8.1 Novelty and positioning

Q1. What exactly is new?
The first FPGA computation of option Greeks by adjoint differentiation, and the first FPGA COS pricer, with (i) a tape-free adjoint datapath at 1.04× the price, (ii) a per-Greek rounding bound for a fixed-point datapath that itself performs the adjoint, and (iii) bit-exact silicon on two FPGA families. Adjoints, COS and Linnainmaa's bound are each established; the combination in hardware is new.

Q2. Is this really AAD?
It is adjoint differentiation; the reverse sweep is hand-derived in the graph and verified. Automatic derivation is future work, and the claim is worded accordingly.

Q3. Hasn't someone done Greeks on FPGAs?
Yes, by re-simulation or finite differences: Klaisoongnoen et al. (HEART 2022, STAC-A2 Monte Carlo on an Alveo U280), the AMD Vitis Quantitative Finance library (finite differences). Capriotti & Giles' 2024 review of fifteen years of AAD in finance lists no FPGA adjoint.

Q4. Cui et al. already derived analytic Heston gradients. Why bother?
We measured that approach: hand-derived Greeks cost 1.5 pricings on a CPU (20.3 μs); the hardware gets 1.04. Our forward-mode study shows the hand-factored forward method needs 25% more multiplies per term than the adjoint.

Q5. Why not just use a CPU?
On a small FPGA a CPU core with analytic Greeks is faster. The FPGA's case is Greeks at almost no extra cost, a certified bound on every Greek, deterministic latency, energy per evaluation, and a narrow latency lead on the larger FPGA.

8.2 Heston and COS

Q6. Which characteristic-function form, and why?
The stable form (Albrecher et al.), $g = (\kappa - i\rho\xi u - d)/(\kappa - i\rho\xi u + d)$ with e^{-dT}; it avoids the branch-cut discontinuity of the complex logarithm for long maturities.

Q7. How did you choose $N = 128$ and the range?
Standard COS choices (cumulant range, ±10 standard deviations). The method-error study shows typical errors around 10^{-9}–10^{-11}, with a few hard cases (high ξ with short T; deep out-of-the-money puts) where 256 terms or a 1.5× wider range is needed.

Q8. You freeze $[a, b]$ when differentiating. Is that valid?

The Greeks are exact derivatives of the COS approximation at a fixed range. The range's dependence on the parameters only affects the truncated tails and vanishes with the truncation error; the reference uses the same convention, and the Greeks were compared with an independent Fourier-integral reference.

Q9. Why puts with parity instead of calls?
Call payoff coefficients contain e^b and cancel catastrophically in fixed point; parity moves that to one exact correction. Worst call price error fell 75×.

Q10. What about the Feller condition?
COS does not need it; the 10,500-input sweep has a Feller-violation regime and the bound held there.

Q11. Your vega is $\partial V/\partial v_0$. Is that standard?
It is with respect to variance; $\partial V/\partial \sigma_0 = 2\sigma_0\, \partial V/\partial v_0$, a one-line conversion.

Q12. Why is S_0 fixed?
Homogeneity: $V(\lambda S_0, \lambda K) = \lambda V(S_0, K)$. The host scales inputs and outputs.

8.3 Adjoint differentiation

Q13. Where is the tape?
There is none. The schedule is static, so each forward value's lifetime is known; the scheduler keeps it in a register until the reverse sweep uses it, within a 506-cycle term (447 storage registers).

Q14. Why reverse mode with only 9 inputs?
Reverse costs about one sweep per output: one output, nine inputs. The forward-mode table quantifies it: 250 multiplies per term against 313–802.

Q15. The cheap-gradient bound is 4–5×. How do you get 1.04?
In operations it is $250/126 \approx 2\times$ multiplies; in *time* it is 1.04 because the CORDIC units set the pace and the adjoint's multiplies fill idle multiplier slots. Where multipliers bind (64-bit) it is 1.55×.

Q16. How did you verify the adjoint?
Against the double-precision model and the forward-mode datapaths (6×10^{-14}), against CoDiPack and analytic Greeks on the CPU (1.3×10^{-10}), and bit-exact RTL against the emulator.

8.4 Fixed point and the bound

Q17. Derive your bound.
Each operation adds a rounding error of at most $\frac{1}{2}$ ulp; to first order the output error is $\sum_i (\partial y/\partial w_i)\, \delta_i$; bound each term by its absolute value: $\sum_i |\partial y/\partial w_i|\, 2^{-(FL+1)}$. The sensitivities come from a reverse sweep over the graph (Linnainmaa 1976).

Q18. Why only first order?
Second-order terms are $\approx 2^{-28}$ smaller. Checked: 50-digit evaluation agrees (≤ 0.24 of the bound), and the bound holds at 44 bits where those terms are 4,096× larger.

Q19. Is it a rigorous bound?
A first-order estimate, not interval arithmetic; rigorous up to the dropped second-order terms, which the checks show are negligible. Say "first-order bound", never "guaranteed".

Q20. Why is the bound loose (0.1–0.4)?
Worst-case half-ulp errors with aligned signs do not happen; rounding errors partly cancel. 0.1–0.4 is tight for a worst-case bound.

Q21. Why 56 bits?
The shortest word at which every Greek's worst relative error is $\leq 1.2 \times 10^{-5}$ on the grid (52 bits: 5.8×10^{-4}), with the error under the bound; and 56×56 maps to 9 DSP48s. The 64-bit design is for the larger chip.

Q22. How do you prevent overflow?
Shift ranges measured over the domain plus margin; every shifter raises `range_err` if exceeded. No overflows over 10,500 inputs; flags only outside the domain after the refit.

Q23. Isn't refitting ranges on test data overfitting?
That is why there is a held-out sweep (seed 20261006, 10,500 new inputs): no in-domain flags, worst bound ratio 0.356.

8.5 Hardware

Q24. Explain modulo scheduling and your II.
Iterations (terms) start every II cycles and overlap; II is at least the resource bound: $\lceil 250/8 \rceil = 32$ for multipliers and 32 for the CORDIC units too. Loop $= (N-1)\cdot\text{II} + \text{latency} = 127\cdot 32+506+2 = 4{,}572$.

Q25. Why no dividers?
Newton–Raphson reciprocals and square roots reuse the multipliers; Tang's exponential, a table-plus-series log and CORDIC (trig, complex magnitude/angle) keep everything on the shared units.

Q26. Why CORDIC rather than DSP-based sin/cos?
CORDIC uses no DSPs (shift-add) and its fixed iteration count suits a static schedule; the price is that it sets the II.

Q27. Why doesn't the Zynq-7020 close 100 MHz?
The 56-bit carry chains; Vivado suggests using the DSP blocks' internal pipeline registers (future work). UltraScale+ meets 100 MHz with +1.6 ns.

Q28. How many resources does the adjoint add?
Same II, same 8 multipliers; extra registers hold forward values for the reverse sweep; still fits the Zynq-7020 (73% LUT).

Q29. How is it verified end to end?
Primitives unit-tested; RTL checked per register against the emulator; wrappers bit-exact; board outputs bit-exact against the emulator, so every model-level accuracy result transfers to silicon. `verify_all.sh` runs 26 checks.

Q30. Did you simulate the gate-level netlist?
Yes, for power: the routed netlists were simulated on 4 evaluations, bit-exact, and the activity fed into Vivado's power analysis (SAIF).

8.6 Results and fairness

Q31. Is the CPU comparison fair?
Same algorithm (COS 128, frozen range, parity) in double precision, g++ -O3 -march=native, best of 5 on an idle machine, and the strongest software baseline (hand-derived analytic Greeks) included. The ZedBoard figure is loop-only (the CPU's work outside its loop is 0.03 μs); bus transfers are not yet timed.

Q32. Your energy numbers?
FPGA: simulated activity on the routed netlist (power $\times$ latency: 0.676 W $\times$ 65.3 μs $\approx$ 44 μJ). CPU: measured package power from HWiNFO (to be added). Until then the energy ratio is an estimate.

Q33. Throughput is worse than 4 cores. Why should anyone care?
The engine uses 20% of the ZCU102's DSPs, so several would fit (not built); per-evaluation latency, determinism and energy are the FPGA's strengths, not batch throughput.

Q34. Only 21 cases for the accuracy figure?
Fig. 6 shows 21 for readability; the claim rests on 10,500 + 10,500 held-out inputs including seven hard regimes, plus a 50-digit check.

Q35. Why are $\partial V/\partial\kappa$ and $\partial V/\partial\xi$ weak?
They are small and computed through long chains with cancellation. Always within their bound, and the bound tells the user their accuracy in advance; that is the point of a bound.

8.7 Practicalities and status

Q36. What happened with the ZCU102 processor run?
The board's DDR fails calibration with Vivado's preset (likely a different memory module on the lab board's revision). The program now runs from on-chip memory and the setup skips the DDR steps; the re-run is pending.

Q37. Why the NIFTY data?
To test silicon on real calibrated parameters, not only random draws: 50 trading days, all bit-exact on the ZCU102.

Q38. Does the hedging show your Greeks are better?
No, and we do not claim that; it is an illustration with 12 trades.

Q39. What's next?
Finish the ZCU102 power/timing run and the CPU power measurement; automatic derivation of the reverse sweep; several engines per device; 100 MHz on the Zynq-7020 with DSP pipeline registers; strike chains (one reverse sweep serves every strike; designed and scheduled, not built).

9 Last-minute advice

- If he challenges a number, answer from `validation/results/` and the paper's tables.
- **Lead with the honest limits** (hand-derived adjoint; first-order bound; simulated FPGA energy and pending CPU power; CPU wins throughput) before he finds them.
- The three sentences to remember:
  1. *"The reverse sweep is compiled into the datapath: there is no tape."*
  2. *"Greeks cost 4% over the price because the reverse sweep fills multiplier slots the CORDIC-bound forward pass leaves idle."*
  3. *"Every Greek has a first-order rounding bound, never exceeded in 21,000 inputs or on silicon."*